

WEEK 8

Aim:

To develop a BusGo Bus Booking web application using HTML, CSS, JavaScript, Node.js, and Express.js with Home, Login, Registration, and User Dashboard pages, and to manage user and booking information using JSON files.

Description:

This experiment demonstrates the development of a basic full-stack bus booking web application using Node.js and Express.js. The application consists of Home, Login, Registration, and Dashboard pages created using HTML and CSS. Express.js is used to create the backend server and handle HTTP requests. User registration details are received from the frontend and stored in a JSON file. During login, the entered username/email and password are compared with the user details stored in the JSON file. The dashboard provides bus search, booking, fare calculation, and booking-history functionality.

Program:

BusGo Bus Booking System Using Node.js and Express.js

Program Code:

```
const express = require("express");
const fs = require("fs");
const path = require("path");
const app = express();
const PORT = 3000;
const USERS_FILE = path.join(__dirname, "users.json");
const BOOKINGS_FILE = path.join(__dirname, "bookings.json");
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, "public")));
```

```

function readJSON(file) {
  try {
    if (!fs.existsSync(file)) fs.writeFileSync(file, "[]");
    return JSON.parse(fs.readFileSync(file, "utf8") || "[]");
  } catch {
    return [];
  }
}

function writeJSON(file, data) {
  fs.writeFileSync(file, JSON.stringify(data, null, 2));
}

app.get("/", (req, res) => {
  res.sendFile(path.join(__dirname, "public", "index.html"));
});

app.post("/register", (req, res) => {
  const { name, age, dob, gender, email, mobile,
    username, password, address } = req.body;
  if (!name || !email || !mobile || !username || !password) {
    return res.status(400).json({
      success: false,
      message: "Please fill all required fields."
    });
  }
  const users = readJSON(USERS_FILE);
  if (users.some(u => u.username.toLowerCase() === username.toLowerCase())) {
    return res.status(409).json({
      success: false,
      message: "Username already exists."
    });
  }
}

```

```

users.push({
  id: Date.now(),
  name, age, dob, gender, email, mobile,
  username, password, address
});
writeJSON(USERS_FILE, users);
res.json({
  success: true,
  message: "Registration successful. You can now login."
});
});

app.post("/login", (req, res) => {
  const { identifier, password } = req.body;
  const users = readJSON(USERS_FILE);
  const user = users.find(
    u => (u.username === identifier || u.email === identifier)
    && u.password === password
  );
  if (!user) {
    return res.status(401).json({
      success: false,
      message: "Invalid username/email or password."
    });
  }
  res.json({
    success: true,
    message: "Login successful.",
    user: {
      id: user.id,
      name: user.name,
      email: user.email,
      username: user.username,
      mobile: user.mobile
    }
  });
});

app.get("/api/buses", (req, res) => {
  res.json([
    { id: 1, operator: "APSRTC Express",
      from: "Visakhapatnam", to: "Vijayawada",
      time: "06:30 AM", price: 520 }
  ]);
});

app.post("/api/bookings", (req, res) => {
  // Receives passenger and bus booking details.
  // Stores the booking in bookings.json.
});

app.listen(PORT, () => {
  console.log("Bus Booking System running at http://localhost:3000");
});

```

● index.html – Home page

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>BusGo - Bus Booking System</title>
<link rel="stylesheet" href="css/style.css">
</head>
<body>
<header>
<nav class="navbar">
<a class="brand" href="index.html">■ BusGo</a>
<div>
<a href="index.html">Home</a>
<a href="register.html">Register</a>
<a href="login.html">Login</a>
</div>
</nav>
</header>
<main>
<section class="hero">
<div>
<span class="badge">SMART BUS BOOKING</span>
<h1>Travel made <span>simple.</span></h1>
<p>Search buses, choose your seats, and book your journey through
a simple and user-friendly bus reservation system.</p>
```

```
<div class="hero-buttons">
<a class="btn primary" href="register.html">Get Started</a>
<a class="btn secondary" href="login.html">Login</a>
</div>
</div>
<div class="bus-art">■</div>
</section>
<section class="features">
<div class="feature">
<b>■ Easy Search</b>
<p>Find available buses by route.</p>
</div>
<div class="feature">
<b>■ Quick Booking</b>
<p>Book your journey in a few steps.</p>
</div>
<div class="feature">
<b>■ Booking History</b>
<p>View your previous reservations.</p>
</div>
</section>
</main>
<footer>© 2026 BusGo – Mini Web Application</footer>
</body>
</html>
```

● Home Page Styling

```
.hero{
min-height:560px;
padding:80px 10%;
display:flex;
align-items:center;
justify-content:space-between;
background:linear-gradient(135deg,#fff,#fff3e8);
}
.hero h1{
font-size:58px;
max-width:650px;
margin:18px 0;
}
.hero h1 span,.welcome span{
color:#e85d04;
}
.btn{
display:inline-block;
border:0;
padding:13px 22px;
border-radius:9px;
font-weight:bold;
cursor:pointer;
}
.primary{background:#e85d04;color:#fff}
.secondary{background:#fff;border:1px solid #ddd}
.features{
display:grid;
grid-template-columns:repeat(3,1fr);
gap:20px;
padding:50px 10%;
}
```

● login.html – Login page

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Login - BusGo</title>
<link rel="stylesheet" href="css/style.css">
</head>
<body>
<header>
<nav class="navbar">
<a class="brand" href="index.html">■ BusGo</a>
<div>
<a href="index.html">Home</a>
<a href="register.html">Register</a>
</div>
</nav>
</header>
<main class="form-page">
<div class="form-card small">
<h2>Welcome back</h2>
<p>Login to manage your bus bookings.</p>
<form id="loginForm">
<label>Username or Email
<input name="identifier" required>
</label>
<label>Password
<input type="password" name="password" required>
</label>
```

```
<button class="btn primary full" type="submit">Login</button>
<p id="message" class="message"></p>
</form>
<p>New user? <a href="register.html">Create an account</a></p>
</div>
</main>
<script src="js/script.js"></script>
</body>
</html>
```


● Login JavaScript

```
function setupLogin() {
  const form = document.getElementById("loginForm");
  const msg = document.getElementById("message");
  form.addEventListener("submit", async e => {
    e.preventDefault();
    const data = Object.fromEntries(new FormData(form).entries());
    try {
      const result = await postJSON("/login", data);
      localStorage.setItem("busUser", JSON.stringify(result.user));
      location.href = "dashboard.html";
    } catch (err) {
      msg.textContent = err.message;
      msg.style.color = "crimson";
    }
  });
}
```

● Login Request / Validation

```
async function postJSON(url, data) {
  const res = await fetch(url, {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify(data)
  });
  const json = await res.json();
  if (!res.ok) throw new Error(json.message || "Request failed");
  return json;
}

// Login accepts username or email.
const user = users.find(
  u => (u.username === identifier || u.email === identifier)
  && u.password === password
);
```

● Login Page Design

```
.form-page{
min-height:calc(100vh - 70px);
display:grid;
place-items:center;
padding:40px;
}
.form-card{
width:min(720px,100%);
background:#fff;
border-radius:15px;
padding:25px;
box-shadow:0 8px 30px #00000009;
}
.form-card.small{max-width:450px}
.form-card h2{margin-top:0;font-size:30px}
input,select,textarea{
border:1px solid #d9dee8;
border-radius:8px;
padding:12px;
font:inherit;
}
```

● Login Flow

The login page sends the entered username/email and password to the Express.js backend using a POST request. The backend reads users.json, validates the credentials, and returns a success or error response. A successful login stores the user information in localStorage and redirects to dashboard.html.

```
POST /login
↓
Express.js receives req.body
↓
Read users.json
↓
Validate username/email + password
↓
Valid? ■■■■■ No ■■■→ Error message
■
Yes
↓
Store logged-in user locally
↓
Open User Dashboard
```

● register.html – Registration page

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Register - BusGo</title>
<link rel="stylesheet" href="css/style.css">
</head>
<body>
<header><nav class="navbar">
<a class="brand" href="index.html">■ BusGo</a>
<div><a href="index.html">Home</a><a href="login.html">Login</a></div>
</nav></header>
<main class="form-page">
<div class="form-card">
<h2>Create your account</h2>
<p>Register to start booking buses.</p>
<form id="registerForm">
```

```
<div class="grid">
<label>Name *<input name="name" required></label>
<label>Age<input type="number" name="age"></label>
<label>Date of Birth<input type="date" name="dob"></label>
<label>Gender<select name="gender">
<option value="">Select</option>
<option>Male</option>
<option>Female</option>
<option>Other</option>
</select></label>
<label>Email *<input type="email" name="email" required></label>
<label>Mobile Number *<input name="mobile" required></label>
<label>Username *<input name="username" required></label>
<label>Password *<input type="password" name="password" required></label>
</div>
<label>Address<textarea name="address" rows="3"></textarea></label>
<button class="btn primary full" type="submit">Register</button>
```

```
<p id="message" class="message"></p>
</form>
<p>Already registered?
<a href="login.html">Login here</a></p>
</div>
</main>
<script src="js/script.js"></script>
</body>
</html>
```

● Registration JavaScript

```
function setupRegister() {
  const form = document.getElementById("registerForm");
  const msg = document.getElementById("message");
  form.addEventListener("submit", async e => {
    e.preventDefault();
    const data = Object.fromEntries(new FormData(form).entries());
    try {
      const result = await postJSON("/register", data);
      msg.textContent = result.message;
      msg.style.color = "green";
      form.reset();
      setTimeout(() => location.href = "login.html", 1200);
    } catch (err) {
      msg.textContent = err.message;
      msg.style.color = "crimson";
    }
  });
}
```


● Registration Data Sent to Backend

```
const data = Object.fromEntries(
  new FormData(form).entries()
);
fetch("/register", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(data)
});
```

● Registration Form Styling

```
.grid{
display:grid;
grid-template-columns:1fr 1fr;
gap:0 18px;
}
.form-card form > label,
.grid label{
display:flex;
flex-direction:column;
gap:7px;
font-weight:bold;
font-size:14px;
margin-top:15px;
}
textarea{resize:vertical}
.full{width:100%;margin-top:12px}
```

● Registration Flow

The user enters personal and account information on the registration page. JavaScript converts the form data into JSON and sends it to POST /register. Express.js receives the data using req.body and appends the new user to users.json without deleting previously registered users.

```
Registration Form
↓
FormData → JSON
↓
POST /register
↓
Express.js / req.body
↓
Read users.json
↓
Append new user
↓
Write users.json
↓
Registration successful
```

● dashboard.html – User Dashboard

Code:

```
<section class="welcome">
<div>
<span class="badge">USER DASHBOARD</span>
<h1>Hello, <span id="userName">Traveler</span> ■</h1>
<p>Plan your next bus journey.</p>
</div>
</section>

<section id="book" class="panel">
<h2>Search & Book a Bus</h2>
<div class="search-grid">
<label>From<select id="from"></select></label>
<label>To<select id="to"></select></label>
<label>Journey Date<input id="date" type="date"></label>
<button class="btn primary" id="searchBtn">
Search Buses
</button>
</div>
<div id="busList" class="bus-list"></div>
</section>
```

```
// Example bus records returned by the backend
```

```
[  
  {  
    "id": 1,  
    "operator": "APSRTC Express",  
    "from": "Visakhapatnam",  
    "to": "Vijayawada",  
    "time": "06:30 AM",  
    "arrival": "01:00 PM",  
    "price": 520,  
    "seats": 18,  
    "type": "AC Seater"  
  },  
  {  
    "id": 2,  
    "operator": "Orange Travels",  
    "from": "Visakhapatnam",  
    "to": "Hyderabad",  
    "time": "08:00 PM",  
    "arrival": "07:00 AM",  
    "price": 950,  
    "seats": 12,  
    "type": "AC Sleeper"  
  }  
]
```

● Booking and History

```
app.post("/api/bookings", (req, res) => {
  const bookings = readJSON(BOOKINGS_FILE);
  const booking = {
    id: "BK" + Date.now(),
    userId: req.body.userId,
    passengerName: req.body.passengerName,
    mobile: req.body.mobile,
    busId: req.body.busId,
    busOperator: req.body.busOperator,
    from: req.body.from,
    to: req.body.to,
    date: req.body.date,
    seats: req.body.seats,
    amount: req.body.amount,
    bookedAt: new Date().toISOString()
  };
  bookings.push(booking);
  writeJSON(BOOKINGS_FILE, bookings);
  res.json({
    success: true,
    message: "Bus booked successfully.",
    booking
  });
});

app.get("/api/bookings/:userId", (req, res) => {
  const bookings = readJSON(BOOKINGS_FILE);
  res.json(bookings.filter(
    b => String(b.userId) === String(req.params.userId)
  ));
});
```

Data: users.json and bookings.json

Initially, the JSON files contain empty arrays. After registration and booking, the corresponding records are appended and persisted.

Example users.json:

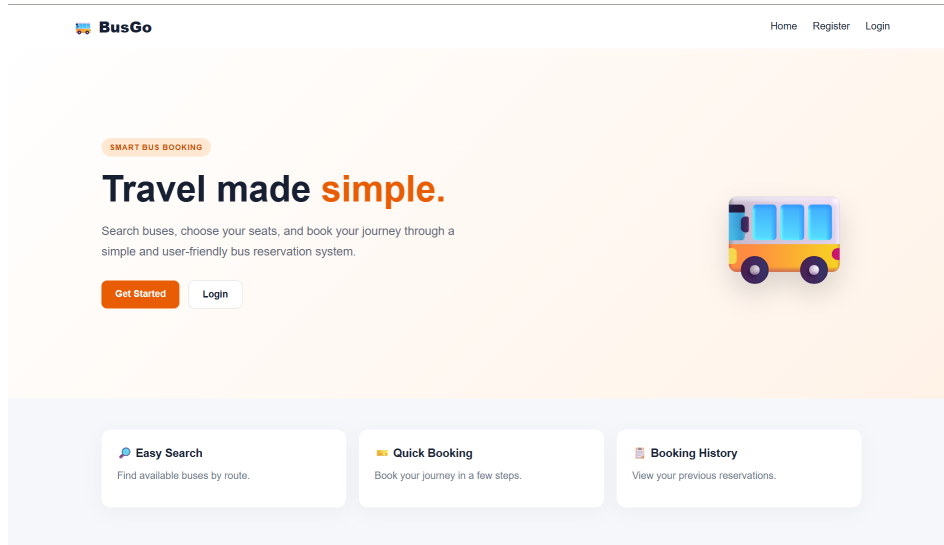
```
[
{
  "id": 17270000000000,
  "name": "AKHIL",
  "age": "20",
  "dob": "2006-05-04",
  "gender": "Male",
  "email": "akhil@gmail.com",
  "mobile": "XXXXXXXXXX",
  "username": "akkiakhil456",
  "password": "*****",
  "address": ""
}
```

Execution of the Program

GET http://localhost:3000 — Displays the BusGo Home page
GET http://localhost:3000/register.html — Displays the Registration page
GET http://localhost:3000/login.html — Displays the Login page
POST /register — Receives registration details and stores them in users.json
POST /login — Checks username/email and password from users.json
GET /api/buses — Returns available bus data
POST /api/bookings — Stores a bus booking in bookings.json
GET /api/bookings/:userId — Displays the user's booking history

Output:

Home Page:



Registration Page:

The screenshot shows the BusGo Registration Page. At the top, there is a navigation bar with the BusGo logo on the left and links for Home and Login on the right. The main content area has a light blue background. In the center, there is a white box with rounded corners titled "Create your account". Below the title, it says "Register to start booking buses." The registration form has several fields: "Name *" (text input with value "AKHIL"), "Age" (text input with value "20"), "Date of Birth" (text input with value "04-05-2006" and a calendar icon), "Gender" (dropdown menu with value "Male"), "Email *" (text input with value "akhil@gmail.com"), "Mobile Number *" (text input with value "6304105258"), "Username *" (text input with value "akkiakhil456"), and "Password *" (password input with value "*****"). Below these fields is a "Address" text area. At the bottom of the form is an orange "Register" button. Below the button, there is a green message that says "Registration successful. You can now login." and a link that says "Already registered? Login here".

Login Page:

BusGo

HomeRegister

Welcome back

Login to manage your bus bookings.

Username or Email

Akkiakhi456

Password

Login

New user? Create an account

User Dashboard / Bus Booking Page:

USER DASHBOARD

Hello, Akhil

Plan your next bus journey.

Search & Book a Bus

From

Any

To

Any

Journey Date

22-09-2026

Search Buses

APSRTC Express

AC Seater

06:30 AM

Arrival 01:00 PM

Visakhapatnam → Vijayawada

18 seats available

₹520

Book

Orange Travels

AC Sleeper

08:00 PM

Arrival 07:00 AM

Visakhapatnam → Hyderabad

12 seats available

₹950

Book

Sri Krishna Travels

AC Sleeper

07:30 PM

Arrival 09:00 AM

Visakhapatnam → Tirupati

20 seats available

₹1100

Book

Morning Star

Non-AC Seater

07:00 AM

Arrival 12:30 PM

Vijayawada → Visakhapatnam

15 seats available

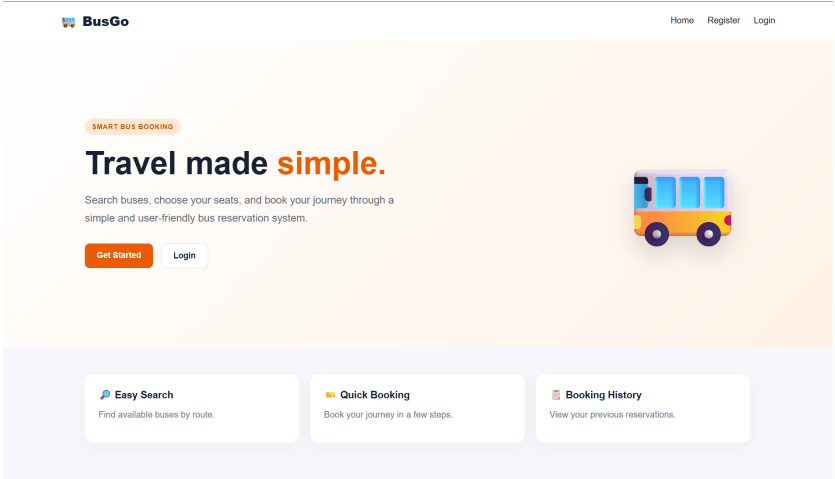
₹500

Book

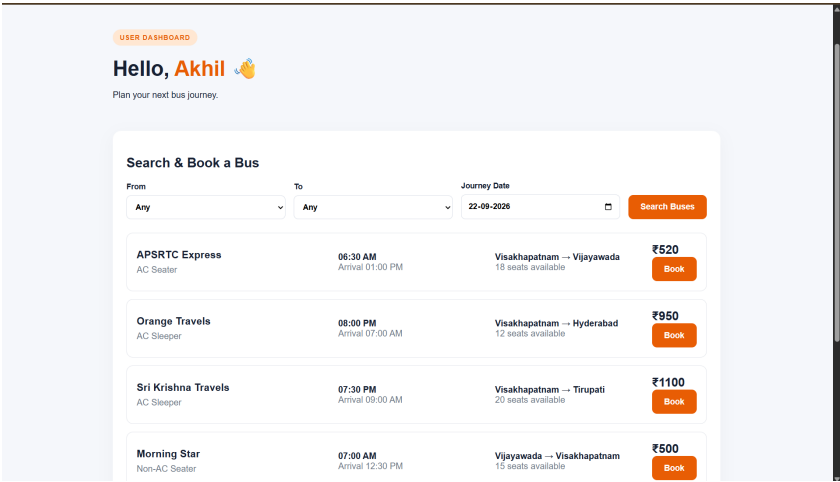
Additional Output Screenshots

The following screenshots demonstrate the completed BusGo interface and the transition from authentication to the main booking dashboard.

Home:



Dashboard:



Result:

The BusGo Bus Booking full-stack web application was successfully developed using HTML, CSS, JavaScript, Node.js, and Express.js. The application demonstrates frontend page creation, Express.js routing, HTTP GET and POST methods, form handling, JSON file operations, user registration, login authentication using JSON-based user data, bus search, booking, fare calculation, and booking-history functionality.

The application successfully connects the frontend pages with the Express.js backend and persists user and booking information in JSON files. The provided output screenshots show the working Home, Registration, Login, and User Dashboard / Bus Booking interfaces.

Technologies Used:

HTML • CSS • JavaScript • Node.js • Express.js • JSON